PHASE 3 OF 18

Control Flow & Arrays

if/else, classic and modern switch, loops, break/continue/labels, keyboard input, arrays, 2D/jagged arrays, the Arrays class and classic array programs

IN THIS PHASE

3.1 Control Statements Overview
3.2 if, if-else and the else-if Ladder
3.3 The switch Statement (classic)
3.4 Modern switch: Expressions and Pattern Matching
3.5 Loops
3.6 break, continue, return and Labels
3.7 Nested Loops and Pattern Programs
3.8 Reading Input from the Keyboard
3.9 Arrays: Basics
3.10 Iterating and Printing Arrays
3.11 The java.util.Arrays Utility Class
3.12 Copying Arrays
3.13 Multidimensional and Jagged Arrays
3.14 Arrays vs ArrayList
3.15 Classic Array Programs
3.16 Predict the Output
3.17 Interview Questions
3.18 Quick Revision

3.1 Control Statements Overview

By default Java executes statements **top to bottom, one after another**. Control statements change that flow based on conditions or repetition.

Category	Statements	Purpose
Decision making (conditional)	`if`, `if-else`, `if-else-if` ladder, nested `if`, `switch`	Choose which block runs
Looping (iteration)	`while`, `do-while`, `for`, enhanced `for` (for-each)	Repeat a block
Branching (jump / transfer)	`break`, `continue`, `return` (+ labels)	Leave or skip part of a loop or method

3.2 if, if-else and the else-if Ladder

```
int age = 20;
if (age >= 18) {                      // 1. if: runs only when the condition is true
    System.out.println("Eligible to vote");
}

int number = 5;
if (number % 2 == 0) {                // 2. if-else: one of two blocks
    System.out.println("Even");
} else {
    System.out.println("Odd");
}

int marks = 75;
if (marks >= 90) {                    // 3. ladder: the FIRST true condition wins
    System.out.println("Grade A");
} else if (marks >= 75) {
    System.out.println("Grade B");    // printed; later branches are skipped
} else if (marks >= 60) {
    System.out.println("Grade C");
} else {
    System.out.println("Fail");
}

if (age >= 18) {                      // 4. nested if
    if (hasLicense) {
        System.out.println("Can drive");
    }
}
```

Rules and traps

```
int a = 10;
if (a) { }          // COMPILE ERROR: int cannot be converted to boolean (no 0/1 truthiness)
if (a = 20) { }     // COMPILE ERROR: assignment yields int, not boolean

boolean flag = false;
if (flag = true) {  // COMPILES! assigns true, then tests it -> always runs (a classic bug)
    System.out.println("Oops");
}

if (a > 5)
    System.out.println("A");
    System.out.println("B");   // NOT part of the if: braces were omitted -> always prints
```

REMEMBER

The `if` condition must be a `boolean` or a boolean expression. Always use braces, even for one-line bodies; it prevents the "second line looks indented but isn't inside" bug.

TIP

Order ladder conditions from most specific to least specific. With `marks >= 60` first, a score of 95 would wrongly get grade C.

3.3 The switch Statement (classic)

`switch` compares one value against several **constant** cases. It is more readable than a long `if-else-if` ladder of equality checks.

```java
int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    case 6:
    case 7:                                  // grouping: several labels share one body
        System.out.println("Weekend");
        break;
    default:                                 // optional; runs when nothing matches
        System.out.println("Invalid day");
}
```

Fall-through

Without `break`, execution **falls through** into the following cases until a `break` or the end of the switch.

```java
int a = 1;
switch (a) {
    case 1: System.out.println("Hi");     // matches
    case 2: System.out.println("Hello");  // falls through
    default: System.out.println("Bye");   // falls through
}
// Output: Hi  Hello  Bye
```

What types can a switch use?

Allowed	Not allowed
`byte`, `short`, `char`, `int` and their wrappers (`Byte`, `Short`, `Character`, `Integer`)	`long`, `float`, `double`, `boolean`
`String` (Java 7+), compared with `equals()`	Ranges such as `case > 10:` (classic switch)
`enum` (Java 5+)	Non-constant case labels (a normal variable)
Any reference type with **pattern matching** (Java 21+)	Duplicate case labels

Rules

1. Case labels must be **compile-time constants** (literals or `final` constants) of a type compatible with the switch value.
2. Case labels must be unique.
3. `default` is optional and can appear anywhere, but put it last by convention.
4. `break` is optional (needed to stop fall-through).
5. Switching on a `null` String, wrapper or enum throws `NullPointerException` in a classic switch.

CORRECTED FROM ORIGINAL NOTES

Two examples in the original notes are wrong:

- `switch (2.1f)` with `case 1.1f:` does **not** fix the error. `float` and `double` are never allowed in a switch.
- `switch ('a') { case 1: ... }` does **not** print "Hi". `'a'` is **97**, not 1, so no case matches and nothing prints. `case 97:` or `case 'a':` would match.

```
String status = "ACTIVE";
switch (status) {                       // String switch (case sensitive)
    case "ACTIVE":   System.out.println("Allow login"); break;
    case "BLOCKED":  System.out.println("Deny");        break;
    default:         System.out.println("Unknown");
}

enum AccountType { SAVINGS, CURRENT, FD }
AccountType type = AccountType.SAVINGS;
switch (type) {
    case SAVINGS: System.out.println("4% interest"); break;   // no "AccountType." prefix
    case CURRENT: System.out.println("0% interest"); break;
    default:      System.out.println("Check FD rates");
}
```

3.4 Modern switch: Expressions and Pattern Matching

MODERN JAVA

Switch expressions (Java 14) use `->` arrows. There is **no fall-through**, no `break`, several labels can share a case, and the switch can **return a value**.

```
int day = 6;
String type = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday";
    case 6, 7          -> "Weekend";
    default            -> "Invalid";
};

int interest = switch (accountType) {   // over an enum, every constant covered:
    case SAVINGS -> 4;                  // no default needed (exhaustive)
    case CURRENT -> 0;
    case FD -> {                        // a block needs 'yield' to return its value
        int base = 6;
        yield base + 1;
    }
};
```

Pattern matching for switch (Java 21)

```
static String describe(Object obj) {
    return switch (obj) {
        case null              -> "null value";            // null can now be handled
        case Integer i when i > 100 -> "large int " + i;   // guard with 'when'
        case Integer i         -> "int " + i;
        case String s          -> "string of length " + s.length();
        case int[] arr         -> "int array of " + arr.length;
        default                -> "other: " + obj.getClass().getSimpleName();
    };
}
```

Feature	Classic `case X:`	Arrow `case X ->`
Fall-through	Yes (needs `break`)	No
Multiple labels	`case 1: case 2:`	`case 1, 2 ->`
Returns a value	No	Yes, as an expression (`yield` inside blocks)
Exhaustiveness checked	No	Yes, when used as an expression
Handles `null`	Throws NPE	`case null ->` (Java 21)

3.5 Loops

A loop repeats a block while a condition holds.

```
int i = 1;
while (i <= 5) {                        // while: entry-controlled, 0 or more runs
    System.out.print(i + " ");
    i++;
}

int j = 1;
do {                                    // do-while: exit-controlled, runs AT LEAST once
    System.out.print(j + " ");
    j++;
} while (j <= 5);                       // note the semicolon

for (int k = 1; k <= 5; k++) {          // for: init; condition; update
    System.out.print(k + " ");
}

int[] arr = {10, 20, 30};
for (int n : arr) {                     // enhanced for (for-each): arrays & Iterables
    System.out.print(n + " ");
}
```

Loop	Control	Minimum runs	Best when
`while`	Entry-controlled	0	Number of iterations unknown (read until end of input, retry until success)
`do-while`	Exit-controlled	1	Body must run at least once (show a menu, then ask again)
`for`	Entry-controlled	0	Count is known; you need the index
for-each	Entry-controlled	0	Reading every element of an array or collection

for-loop variations

```
for (int i = 0, j = 10; i < j; i++, j--) { }   // several variables and updates
for (;;) { }                                    // all parts optional = infinite loop
for (int i = 10; i > 0; i -= 2) { }             // counting down in steps of 2
```

Limitations of for-each

- No access to the index.
- Assigning to the loop variable does **not** change the array: `for (int n : arr) n = 0;` leaves `arr` unchanged.
- Removing from a collection inside for-each throws `ConcurrentModificationException`. Use an `Iterator` or `removeIf()` (Phase 13).
- Only goes forward, over one collection at a time.

Infinite loops and unreachable code

```
while (true) { System.out.println("Hi"); }
System.out.println("Hello");   // COMPILE ERROR: unreachable statement

while (false) { System.out.println("Hi"); }   // COMPILE ERROR: unreachable statement
System.out.println("Hello");

if (false) { System.out.println("Hi"); }      // compiles: if(false) is allowed
                                              // (used for conditional compilation)
```

3.6 break, continue, return and Labels

Statement	Effect
`break`	Exits the **nearest** loop or switch immediately
`continue`	Skips the rest of the current iteration and goes to the next one
`return`	Exits the whole **method** (optionally returning a value)
`break label` / `continue label`	Applies to an **outer** labelled loop

```
for (int i = 1; i <= 10; i++) {
    if (i == 3) continue;    // skip 3
    if (i == 7) break;       // stop the loop at 7
    System.out.print(i + " ");
}
// Output: 1 2 4 5 6
```

```
outer:                                  // label: identifier followed by ':'
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == 2) continue outer;     // jump to the next i
        if (i == 3) break outer;        // leave BOTH loops
        System.out.println(i + "," + j);
    }
}
// Output: 1,1  2,1
```

TIP

Labelled breaks are handy for searching a 2D grid, but if you need them often, extract the loops into a method and `return` instead. It reads better.

3.7 Nested Loops and Pattern Programs

```
// Multiplication table
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 5; j++) {
        System.out.print(i * j + "\t");
    }
    System.out.println();
}

int n = 4;
for (int i = 1; i <= n; i++) {          // right-angled triangle
    System.out.println("* ".repeat(i)); // *  /  * *  /  * * *  /  * * * *
}

for (int i = 1; i <= n; i++) {          // pyramid: spaces, then 2i-1 stars
    System.out.println(" ".repeat(n - i) + "*".repeat(2 * i - 1));
}
//    *
//   ***
//  *****
// *******
```

TIP

For pattern questions, the outer loop is almost always the **row** and the inner loop(s) control the **columns** (spaces first, then symbols). Work out the formula for row `i` before coding.

3.8 Reading Input from the Keyboard

```
import java.io.*;

public class Eligibility {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        System.out.print("Enter age: ");
        int age = Integer.parseInt(br.readLine().trim());
        System.out.print("Enter salary: ");
        double salary = Double.parseDouble(br.readLine().trim());
        System.out.println(age >= 21 && salary > 25000 ? "Eligible" : "Not eligible");
    }
}
```

```
System.in (bytes) -> InputStreamReader (bytes -> chars) -> BufferedReader (buffered lines)
```

Option	Package	Speed	Parses numbers?	Checked exception?	Best for
`BufferedReader`	`java.io`	Fast (buffered)	No: use `parseInt`	Yes, `IOException`	Large input, competitive programming
`Scanner`	`java.util`	Slower (regex parsing)	Yes: `nextInt()`	No	Simple interactive programs
Command-line args	`String[] args`	Instant	No: use `parseInt`	No	Values known at launch
`Console`	`java.io`	n/a	No	No	Reading passwords (`readPassword()`)

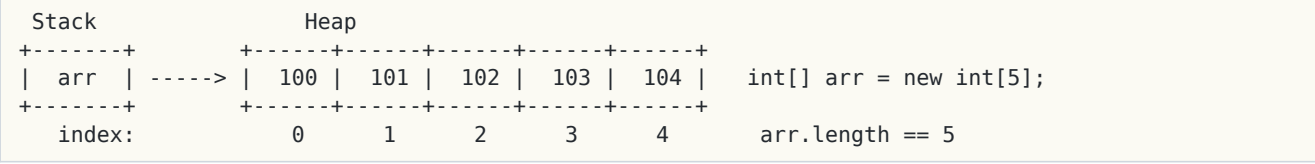
3.9 Arrays: Basics

An **array** is a fixed-size container of elements of the **same type**, stored in **contiguous** memory and accessed by a zero-based **index**.

- Arrays are **objects** (a reference type), so they live on the heap.
- Size is fixed once created; use `ArrayList` when you need to grow.
- Elements are homogeneous (same type). An `Object[]` can hold anything, since everything is an `Object`.
- The `length` field gives the size. It is a **field**, not a method: `arr.length`, not `arr.length()`.

```
int[] a;                // declaration (preferred style)
int b[];                // C-style, also valid
a = new int[5];         // creation: 5 ints, all 0
int[] c = new int[5];   // declaration + creation

int[] d = {10, 20, 30};             // array literal (only at declaration)
int[] e = new int[]{10, 20, 30};    // anonymous array: works anywhere
int[] f;
// f = {1, 2};                      // COMPILE ERROR: literal only allowed at declaration
f = new int[]{1, 2};                // OK
```



Default values of array elements

Element type	Default
`byte`, `short`, `int`, `long`	`0`
`float`, `double`	`0.0`
`char`	`'\u0000'`
`boolean`	`false`
Any reference (`String`, objects, inner arrays)	`null`

Size rules

```
int[] a = new int[0];        // OK: empty array (length 0), useful as a "no results" value
int n = 5; int[] b = new int[n * 2];   // OK: size may be any int expression
int[] c = new int['A'];      // OK: char promoted to int (65)
int[] d = new int[4.02];     // COMPILE ERROR: possible lossy conversion from double
int[] e = new int[5L];       // COMPILE ERROR: long not allowed
int[] f = new int[-4];       // compiles, but RUNTIME NegativeArraySizeException
System.out.println(c[65]);   // RUNTIME ArrayIndexOutOfBoundsException (valid: 0..64)
```

NOTE

The compiler doesn't know an array's size, because arrays are created at runtime. That is why a bad index is not a compile error: it throws `ArrayIndexOutOfBoundsException` at runtime. Valid indexes are `0` to `length - 1`.

3.10 Iterating and Printing Arrays

```
import java.util.Arrays;

int[] arr = {10, 20, 30};
for (int i = 0; i < arr.length; i++) System.out.print(arr[i] + " ");  // with index
for (int x : arr) System.out.print(x + " ");                          // for-each
System.out.println(Arrays.toString(arr));    // [10, 20, 30]
Arrays.stream(arr).forEach(System.out::println);   // streams (Phase 15)

System.out.println(arr);     // [I@15db9742  <- type code + hash, NOT the contents
```

NOTE

In `[I@15db9742`, `[` means array and `I` means int. Other codes are `[D` for double, `[Ljava.lang.String;` for String[] and `[[I` for int[][]. The part after `@` is the hash code in hex. Use `Arrays.toString()` for 1D arrays and `Arrays.deepToString()` for 2D arrays.

3.11 The java.util.Arrays Utility Class

Method	Purpose	Example
`toString(arr)`	Readable String of a 1D array	`[10, 20, 30]`
`deepToString(arr)`	Readable String of nested arrays	`[[1, 2], [3, 4]]`
`sort(arr)`	Sorts ascending (dual-pivot quicksort for primitives, TimSort for objects)	`Arrays.sort(arr);`
`sort(arr, from, to)`	Sorts a range (`to` exclusive)	`Arrays.sort(arr, 0, 3);`
`sort(objArr, comparator)`	Custom order (objects only)	`Arrays.sort(names, Comparator.reverseOrder());`
`binarySearch(arr, key)`	Index of key in a **sorted** array; negative if absent	`Arrays.binarySearch(arr, 20);`
`equals(a, b)` / `deepEquals(a, b)`	Compares contents (1D / nested)	`Arrays.equals(a1, a2)`
`fill(arr, val)`	Sets every element to `val`	`Arrays.fill(arr, -1);`
`copyOf(arr, newLen)`	New array, truncated or padded with defaults	`Arrays.copyOf(arr, 5);`
`copyOfRange(arr, from, to)`	Copies part (`to` exclusive)	`Arrays.copyOfRange(arr, 1, 4);`
`asList(...)`	Fixed-size `List` view backed by the array	`Arrays.asList("a", "b")`
`stream(arr)`	Stream of the elements	`Arrays.stream(arr).sum()`
`hashCode(arr)`	Content-based hash	`Arrays.hashCode(arr)`

```
int[] nums = {50, 10, 40, 20, 30};
Arrays.sort(nums);                                  // [10, 20, 30, 40, 50]
int pos = Arrays.binarySearch(nums, 40);            // 3
int sum = Arrays.stream(nums).sum();                // 150
int max = Arrays.stream(nums).max().getAsInt();     // 50

Integer[] boxed = {5, 1, 4};
Arrays.sort(boxed, Collections.reverseOrder());     // [5, 4, 1] (needs Integer[], not int[])

int[] a1 = {1, 2}, a2 = {1, 2};
System.out.println(a1 == a2);                 // false: different objects
System.out.println(a1.equals(a2));            // false: arrays don't override equals()!
System.out.println(Arrays.equals(a1, a2));    // true:  compares contents
```

PITFALL

`Arrays.asList()` returns a **fixed-size** list: `add()` and `remove()` throw `UnsupportedOperationException`, and changes to it write through to the array. For a real list use `new ArrayList<>(Arrays.asList(...))` or `List.of(...)` (immutable). Also, `Arrays.asList(intArray)` gives a `List<int[]>` with one element, not a list of numbers.

3.12 Copying Arrays

```
int[] src = {1, 2, 3, 4, 5};

int[] ref = src;                                // NOT a copy: both names, one array
ref[0] = 99;                                    // src[0] is now 99 too!

int[] c1 = src.clone();                         // new array, same elements
int[] c2 = Arrays.copyOf(src, src.length);      // same, can resize
int[] c3 = Arrays.copyOfRange(src, 1, 3);       // [2, 3]
int[] c4 = new int[5];
System.arraycopy(src, 0, c4, 0, src.length);    // (src, srcPos, dest, destPos, length)
```

> **NOTE**
> All of these are **shallow** copies. For an array of objects (or a 2D array), the new array points to the **same** inner objects or rows. A deep copy must copy each element or row (see object cloning in Phase 8).

3.13 Multidimensional and Jagged Arrays

A 2D array is an **array of arrays**: each row is a separate 1D array object.

```
int[][] m = {{100, 200}, {300, 400}};

 m ---> [ row0 ref | row1 ref ]
            |           |
            v           v
       [100, 200]   [300, 400]      m.length = 2 rows, m[0].length = 2 columns
```

```
int[][] m = new int[2][3];        // 2 rows x 3 columns, all 0
m[0][1] = 200;
int[][] lit = { {100, 200}, {300, 400} };

for (int i = 0; i < lit.length; i++) {             // index-based
    for (int j = 0; j < lit[i].length; j++) {      // use lit[i].length, not lit[0]
        System.out.print(lit[i][j] + " ");
    }
    System.out.println();
}
for (int[] row : lit) {                            // for-each
    System.out.println(Arrays.toString(row));
}
System.out.println(Arrays.deepToString(lit));      // [[100, 200], [300, 400]]
```

Jagged arrays (rows of different lengths)

```
int[][] jagged = new int[3][];   // only the number of rows is fixed
jagged[0] = new int[]{1};
jagged[1] = new int[]{1, 2};
jagged[2] = new int[]{1, 2, 3};
// Pascal's triangle, seating plans, monthly data with different day counts...

int[][] partial = new int[2][];
System.out.println(partial[0]);      // null: the rows were never created
System.out.println(partial[0][0]);   // NullPointerException
// int[][] bad = new int[][3];       // COMPILE ERROR: first dimension is required

int[][][] cube = new int[2][3][4];   // 3D: 2 blocks x 3 rows x 4 cols
```

3.14 Arrays vs ArrayList

Feature	Array	ArrayList
Size	Fixed at creation	Grows and shrinks automatically
Element types	Primitives **and** objects	Objects only (`Integer`, not `int`; autoboxing)
Length	`arr.length` (field)	`list.size()` (method)
Performance	Slightly faster, no boxing	Small overhead
Utilities	`Arrays` class	Rich `List` API: `add`, `remove`, `contains`, ...
Generics / type safety	Covariant (`Object[] o = new String[1]` compiles)	Invariant, checked at compile time
Multidimensional	Built in (`int[][]`)	List of lists

PITFALL

Array **covariance**: `Object[] objs = new String[2]; objs[0] = 10;` compiles but throws `ArrayStoreException` at runtime. Generic collections catch this at compile time.

3.15 Classic Array Programs

```
// 1. Reverse in place (two pointers)
int[] arr = {10, 20, 30, 40, 50};
for (int s = 0, e = arr.length - 1; s < e; s++, e--) {
    int t = arr[s]; arr[s] = arr[e]; arr[e] = t;
}
// [50, 40, 30, 20, 10]

// 2. Max, min, sum, average in one pass
int max = arr[0], min = arr[0], sum = 0;
for (int x : arr) {
    if (x > max) max = x;
    if (x < min) min = x;
    sum += x;
}
double avg = (double) sum / arr.length;   // cast first, or integer division

// 3. Second largest (single pass, handles duplicates)
int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
for (int x : new int[]{5, 9, 9, 3, 7}) {
    if (x > first) { second = first; first = x; }
    else if (x > second && x != first) second = x;
}
// first = 9, second = 7
```

```
// 4. Non-repeated elements: O(n^2) nested loop (from the original notes)
int[] a = {1, 2, 3, 2, 4, 5, 1};
for (int i = 0; i < a.length; i++) {
    int count = 0;
    for (int j = 0; j < a.length; j++) {
        if (a[i] == a[j] && i != j) count++;
    }
    if (count == 0) System.out.print(a[i] + " ");   // 3 4 5
}

// 4b. Same in O(n) with a HashMap: the answer interviewers hope for
Map<Integer, Integer> freq = new LinkedHashMap<>();
for (int x : a) freq.merge(x, 1, Integer::sum);
freq.forEach((k, v) -> { if (v == 1) System.out.print(k + " "); });
```

```java
// 5. Binary search: O(log n), array must be sorted
static int binarySearch(int[] a, int key) {
    int low = 0, high = a.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;   // avoids int overflow of (low + high)
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1; else high = mid - 1;
    }
    return -1;
}

// 6. Bubble sort: O(n^2), stops early when already sorted
static void bubbleSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < a.length - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

```java
// 7. Matrix addition and transpose
int[][] x = {{1, 2}, {3, 4}}, y = {{5, 6}, {7, 8}};
int[][] add = new int[2][2], tr = new int[2][2];
for (int i = 0; i < 2; i++)
    for (int j = 0; j < 2; j++) {
        add[i][j] = x[i][j] + y[i][j];   // [[6, 8], [10, 12]]
        tr[j][i] = x[i][j];              // [[1, 3], [2, 4]]
    }
```

3.16 Predict the Output

```java
int x = 5;
if (x > 2) if (x > 10) System.out.println("A"); else System.out.println("B");
// B : 'else' binds to the NEAREST if (dangling else)

int i = 0;
while (i++ < 3) System.out.print(i + " ");     // 1 2 3
int j = 0;
do { j++; } while (j < 0);  System.out.println(j);    // 1 (runs once)

for (int k = 0; k < 3; k++);                   // stray semicolon: empty loop body
{ System.out.println("once"); }                // runs ONCE, not 3 times

int[] arr = new int[3];
System.out.println(arr[1] + " " + arr.length);  // 0 3
String[] s = new String[2];
System.out.println(s[0]);                        // null
System.out.println(s[0].length());               // NullPointerException

char c = 'b';
switch (c) { case 'a': System.out.print("A"); case 'b': System.out.print("B");
             case 'c': System.out.print("C"); }  // BC (fall-through)
```

3.17 Interview Questions

Q. What is the difference between `while` and `do-while`?

`while` checks the condition before each iteration and may run zero times. `do-while` checks after, so its body runs at least once.

Q. Which types can be used in a switch?

`byte`, `short`, `char`, `int`, their wrappers, `String` (Java 7) and `enum`. From Java 21, any reference type through pattern matching. Never `long`, `float`, `double` or `boolean`.

Q. What is fall-through in a switch?

Without `break`, execution continues into the following case bodies. Arrow-style (`->`) cases never fall through.

Q. What is the difference between a switch statement and a switch expression?

A switch expression (Java 14) produces a value, uses `->` with no fall-through, allows comma-separated labels, uses `yield` inside blocks, and must be exhaustive.

Q. `break` vs `continue` vs `return`?

`break` exits the nearest loop or switch; `continue` skips to the next iteration; `return` exits the entire method. Labelled `break`/`continue` target an outer loop.

Q. Is an array a primitive or an object? Where is it stored?

Always an object, even `int[]`. It is created on the heap, and the variable holds a reference.

Q. Why is `length` not a method for arrays but `length()` is for Strings?

Array `length` is a final field set when the array is created. `String.length()` is a method of the `String` class.

Q. What happens with `int[] a = new int[-1];` and with `a[5]` on a 5-element array?

The first compiles but throws `NegativeArraySizeException` at runtime. The second throws `ArrayIndexOutOfBoundsException`. Both are runtime errors because sizes and indexes are checked when the code runs.

Q. How do you compare two arrays' contents?

`Arrays.equals(a, b)` (or `Arrays.deepEquals` for nested arrays). `==` and `a.equals(b)` both compare references.

Q. What is a jagged array?

A multidimensional array whose rows have different lengths, created by specifying only the first dimension (`new int[3][]`) and then each row separately.

Q. How do you copy an array?

`clone()`, `Arrays.copyOf()`, `Arrays.copyOfRange()` or `System.arraycopy()`. All are shallow copies; plain assignment copies only the reference.

3.18 Quick Revision

- Conditions must be `boolean`; no `if (1)`. Always use braces.
- Ladder: the first true branch wins. `else` binds to the nearest `if`.
- Switch types: byte/short/char/int (+ wrappers), String, enum, patterns (21). **No** long/float/double/boolean.
- Classic switch falls through without `break`; arrow switch doesn't, and can return a value (`yield` in blocks).
- `while` runs 0+ times, `do-while` 1+ times, `for` when the count is known, for-each for read-only traversal.
- `while (true)` / `while (false)` followed by code: unreachable statement compile error.
- Labelled `break outer;` exits nested loops.
- Arrays: fixed size, same type, heap objects, index `0..length-1`, `length` is a field.

- Element defaults: 0 / 0.0 / `'\u0000'` / false / null.
- Bad index → `ArrayIndexOutOfBoundsException`; negative size → `NegativeArraySizeException` (both at runtime).
- Print with `Arrays.toString` / `deepToString`; compare with `Arrays.equals`.
- Copies are shallow; `Arrays.asList` is fixed-size.
- 2D arrays are arrays of arrays; jagged rows are allowed; `new int[2][]` rows are `null`.